

PROGRAMACIÓN II

Trabajo Práctico 3: Introducción a la Programación Orientada a Objetos

OBJETIVO GENERAL

Comprender los fundamentos de la Programación Orientada a Objetos, incluyendo clases, objetos, atributos y métodos, para estructurar programas de manera modular y reutilizable en Java.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Clases y Objetos	Modelado de entidades como Estudiante, Mascota, Libro, Gallina y NaveEspacial
Atributos y Métodos	Definición de propiedades y comportamientos para cada clase
Estado e Identidad	Cada objeto conserva su propio estado (edad, calificación, combustible, etc.)
Encapsulamiento	Uso de modificadores de acceso y getters/setters para proteger datos
Modificadores de acceso	Uso de private, public y protected para controlar visibilidad
Getters y Setters	Acceso controlado a atributos privados mediante métodos
Reutilización de código	Definición de clases reutilizables en múltiples contextos

Caso Práctico

Desarrollar en Java los siguientes ejercicios aplicando los conceptos de programación orientada a objetos:

1. Registro de Estudiantes

- Crear una clase Estudiante con los atributos: nombre, apellido, curso, calificación.

Métodos requeridos: **mostrarInfo()**, **subirCalificacion(puntos)**, **bajarCalificacion(puntos)**.

Tarea: Instanciar a un estudiante, mostrar su información, aumentar y disminuir calificaciones.

```
package trabajo.practico.pkg1;

public class Estudiantes {
    String nombre;
    String apellido;
    String curso;
    double calificacion;
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getApellido() {
        return apellido;
    }
    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
    public String getCurso() {
        return curso;
    }
    public void setCurso(String curso) {
        this.curso = curso;
    }
    public double getCalificacion() {
        return calificacion;
    }
    public void setCalificacion(double calificacion) {
        this.calificacion = calificacion;
    }
    void mostrarInfo() {
        System.out.println("Nombre: " + nombre + "Apellido: " + apellido + "Curso: " + curso + "Calificacion: " + calificacion);
    }
    public void subirCalificacion(double puntos) {
        calificacion += puntos;
    }
    public void bajarCalificacion(double puntos) {
        calificacion -= puntos;
    }
}
```

```
package trabajo.practico.pkg3;
public class TrabajoPractico3 {
    public static void main(String[] args) {
        // Crear un objeto de clase estudiante.
        Estudiantes estudiante1 = new Estudiantes();
        // Instanciar un objeto de clase estudiante.
        estudiante1.setNombre("Lorenzo");
        estudiante1.setApellido("Blanco");
        estudiante1.setCurso("1 A");
        estudiante1.setCalificacion(9);
        estudiante1.mostrarInfo();
        // Subir calificación utilizando el metodo subirCalificacion(puntos)
        estudiante1.subirCalificacion(1);
        System.out.println("La calificación de " + estudiante1.nombre + " es " + estudiante1.calificacion);
        // Bajar calificación utilizando el metodo bajarCalificacion(puntos)
        estudiante1.bajarCalificacion(1);
        System.out.println("La calificación de " + estudiante1.nombre + " es " + estudiante1.calificacion);
    }
}
```

```
run:
Nombre: Lorenzo
Apellido: Blanco
Curso: 1 A
Calificacion: 9.0
La calificacion de Lorenzo es 10.0
La calificacion de Lorenzo es 9.0
```

2. Registro de Mascotas
 - a. Crear una clase Mascota con los atributos: nombre, especie, edad.

Métodos requeridos: [mostrarInfo\(\)](#), [cumplirAños\(\)](#).

Tarea: Crear una mascota, mostrar su información, simular el paso del tiempo y verificar los cambios.

```
package trabajo.practico.pkg3;

public class Mascotas {
    String nombre;
    String especie;
    int edad;
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getEspecie() {
        return especie;
    }
    public void setEspecie(String especie) {
        this.especie = especie;
    }
    public int getEdad() {
        return edad;
    }
    public void setEdad(int edad) {
        this.edad = edad;
    }
    void mostrarInfo(){
        System.out.println("Nombre: " + nombre + "\nEspecie: " + especie + "\nEdad: " + edad);
    }
    void cumplirAños(){
        edad += 1;
    }
}
```

```
// Crear un objeto de la clase Mascotas.  
Mascotas mascotal = new Mascotas();  
// Instanciar un objeto de la clase Mascotas.  
mascotal.setNombre("Enock");  
mascotal.setEspecie("Maltes");  
mascotal.setEdad(11);  
  
// Mostrar informacion.  
mascotal.mostrarInfo();  
// Simular el paso del tiempo.  
mascotal.cumplirAños();  
// Mostrar edad actualizada.  
mascotal.mostrarInfo();
```

```
Nombre: Enock  
Especie: Maltes  
Edad: 11  
Nombre: Enock  
Especie: Maltes  
Edad: 12
```

3. Encapsulamiento con la Clase Libro
 - a. Crear una clase Libro con atributos privados: titulo, autor, añoPublicacion.

Métodos requeridos: Getters para todos los atributos. Setter con validación para añoPublicacion.

Tarea: Crear un libro, intentar modificar el año con un valor inválido y luego con uno válido, mostrar la información final.

```
package trabajo.practico.pkg3;
public class Libro {
    private String titulo;
    private String autor;
    private int anioPublicacion;

    public String getTitulo() {
        return titulo;
    }
    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }
    public String getAutor() {
        return autor;
    }
    public void setAutor(String autor) {
        this.autor = autor;
    }
    public int getAnioPublicacion() {
        return anioPublicacion;
    }
    public void setAnioPublicacion(int anioPublicacion) {
        if (anioPublicacion <= 2025) {
            this.anioPublicacion = anioPublicacion;
        } else {
            System.out.println("El año ingresado es invalido. ");
        }
    }
    void mostrarInfo() {
        System.out.println("Libro: " + titulo + "\nAutor: " + autor + "\nAño Publicacion: " + anioPublicacion);
    }
}
```

```
// Crear un objeto de la clase Libro.
Libro librol = new Libro();

// Instanciar un objeto de la clase Libro.
librol.setTitulo("El hombre mas rico de babilonia.");
librol.setAutor("George Samuel Clason");
librol.setAnioPublicacion(3000);
// Mostrar informacion con anio valido.
librol.setAnioPublicacion(1926);
librol.mostrarInfo();
```

```
El año ingresado es invalido.
Libro: El hombre mas rico de babilonia.
Autor: George Samuel Clason
Año Publicacion: 1926
```

4. Gestión de Gallinas en Granja Digital
 - a. Crear una clase Gallina con los atributos: idGallina, edad, huevosPuestos.

Métodos requeridos: `ponerHuevo()`, `envejecer()`, `mostrarEstado()`.

Tarea: Crear dos gallinas, simular sus acciones (envejecer y poner huevos), y mostrar su estado.

```
package trabajo.practico.pkg3;

public class Gallina {
    private int idGallina;
    private int edad;
    private int huevosPuestos;
    public int getIdGallina() {
        return idGallina;
    }
    public void setIdGallina(int idGallina) {
        this.idGallina = idGallina;
    }
    public int getEdad() {
        return edad;
    }
    public void setEdad(int edad) {
        this.edad = edad;
    }
    public int getHuevosPuestos() {
        return huevosPuestos;
    }
    public void setHuevosPuestos(int huevosPuestos) {
        this.huevosPuestos = huevosPuestos;
    }
    void ponerHuevos(int huevos) {
        huevosPuestos += huevos;
    }
    void envejecer(int años) {
        if (años > 0) {
            edad += años;
        }
    }
    void mostrarEstado() {
        System.out.println("Gallina " + idGallina + "\nEdad: " + edad + "\nHuevos Puestos: " + huevosPuestos);
    }
}
```

```
// Crear dos objeto de clase Gallina.  
Gallina gallinal = new Gallina();  
Gallina gallina2 = new Gallina();  
  
// Instanciamos los objetos de la clase Gallina.  
gallinal.setIdGallina(1);  
gallinal.setEdad(10);  
gallinal.setHuevosPuestos(100);  
gallinal.mostrarEstado();  
  
gallina2.setIdGallina(2);  
gallina2.setEdad(5);  
gallina2.setHuevosPuestos(50);  
gallina2.mostrarEstado();  
  
// Utilizamos los metodos de la clase Gallina.  
gallinal.ponerHuevos(10);  
gallinal.envejecer(1);  
gallinal.mostrarEstado();  
  
gallina2.ponerHuevos(10);  
gallina2.envejecer(1);  
gallina2.mostrarEstado();
```

```
Gallina 1  
Edad: 10  
Huevos Puestos: 100  
Gallina 2  
Edad: 5  
Huevos Puestos: 50  
Gallina 1  
Edad: 11  
Huevos Puestos: 110  
Gallina 2  
Edad: 6  
Huevos Puestos: 60
```


5. Simulación de Nave Espacial

Crear una clase NaveEspacial con los atributos: nombre, combustible.

Métodos requeridos: `despegar()`, `avanzar(distancia)`, `recargarCombustible(cantidad)`, `mostrarEstado()`.

Reglas: Validar que haya suficiente combustible antes de avanzar y evitar que se supere el límite al recargar.

Tarea: Crear una nave con 50 unidades de combustible, intentar avanzar sin recargar, luego recargar y avanzar correctamente. Mostrar el estado al final.

```
package trabajo.practico.pkg3;
public class NaveEspacial {
    private String nombre;
    private int combustible;
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public double getCombustible() {
        return combustible;
    }
    public void setCombustible(int combustible) {
        this.combustible = combustible;
    }
    void despegar() {
        System.out.println("La nave espacial esta despegando...");
    }
    void avanzar(double distancia) {
        if (combustible > 50) {
            System.out.println("Avanzando...");
        } else {
            System.out.println("La cantidad de combustible no es suficiente para avanzar. ");
        }
    }
    void recargarCombustible(double recarga) {
        if ((combustible + recarga) < 50) {
            System.out.println("La recarga no es posible, es menor al minimo del tanque. ");
        } else if ((combustible + recarga) > 1000) {
            System.out.println("La carga no es posible, excede la capacidad del tanque. ");
        } else {
            combustible += recarga;
            System.out.println("Combustible cargado correctamente...");
        }
    }
    void mostrarEstado() {
        System.out.println("Nave: " + nombre + "\nCombustible: " + combustible);
    }
}
```



```
// Crear un objeto de clase NaveEspacial.  
NaveEspacial navel = new NaveEspacial();  
// Instanciamos un objeto de la clase NaveEspacial.  
navel.setNombre("Apolo XV");  
navel.setCombustible(50);  
navel.mostrarEstado();  
  
navel.despegar();  
navel.avanzar(100);  
  
navel.recargarCombustible(100);  
  
navel.mostrarEstado();  
navel.avanzar(100);  
navel.mostrarEstado();
```

```
Nave: Apolo XV  
Combustible: 50  
La nave espacial esta despegando...  
La cantidad de combustible no es sufuciente para avanzar.  
Combustible cargado correctamente...  
Nave: Apolo XV  
Combustible: 150  
Avanzando...  
Nave: Apolo XV  
Combustible: 150  
BUILD SUCCESSFUL (total time: 0 seconds)
```

CONCLUSIONES ESPERADAS

- Comprender la diferencia entre clases y objetos.
- Aplicar principios de encapsulamiento para proteger los datos.
- Usar getters y setters para gestionar atributos privados.
- Implementar métodos que definen comportamientos de los objetos.
- Manejar el estado y la identidad de los objetos correctamente.
- Aplicar buenas prácticas en la estructuración del código orientado a objetos.
- Reforzar el pensamiento modular y la reutilización del código en Java.

Link al repositorio de Github:
<https://github.com/LorenzoBlanco1/ProgramacionII>